

AI ASSIGNMENT 1: UI/UX & SYSTEM SPECIFICATION

Group Code: AI_A1_G08 | Course: SWE3513 - Artificial Intelligence

1. Executive Summary & Problem Statement

Smallholder farmers and agricultural managers often face severe yields unpredictability due to erratic weather, soil acidity variations, and logistical delays. Standard machine learning solutions provide numerical outputs, but often lack human-centered interpretability and input validation, leading to poor operational decisions.

Objective: Design an end-to-end Machine Learning interface (CLI and API schema) that predicts farm productivity, categorizes yields into risk buckets, and clusters operational regions while enforcing strict input boundary checks and transparent Responsible AI feedback loops.

2. Primary User Personas & Target Audience

User Role	Primary Goal	Key Technical Needs	Pain Points
Agronomist Lead	Analyze historical yield predictors	Batch inference, JSON output	Black-box models without metrics
Field Coordinator	Predict yield per plot on real data	CLI interface, immediate errors	Cryptic stack traces on invalid data
System Auditor	Verify model metrics and artifacts	Traceable evaluation logs	Unverified parameters or missing model versions

3. End-to-End User Journey

The diagram below outlines the interaction pipeline for field users executing predictions via the system CLI interface.

Phase	User Action	System Processing	Output / Feedback
1. Initialization	Runs predict.py with JSON payload	loads model JSON/joblib models & schemas	Models validated in memory
2. Input Parsing	Passes raw JSON record parameters	Schema validation & missing key audit	Returns structured error if invalid
3. Preprocessing	Data enters pipeline	StandardScaling applied to input features	Features normalized
4. Multi-Model Inference	Inference triggered	Regression + Classification + Clustering	Predictions computed concurrently
5. Structuring	Formatter processes results	JSON response built with version & graph	Standardized JSON printed to stdout

4. Wireframe Layout 1: Input CLI Payload Schema

Annotated conceptual wireframe showing the structure and constraints of input payload passing into `predict.py`.

[CLI Wireframe Box - Input Interface]	Annotations & Specifications
<pre>\$ python predict.py --record '{ "plot_area_ha": 1.2, "rainfall_mm": 81, "soil_ph": 5.7, "seed_kg": 210, "distance_km": 14, "arrival_hour": 9 }'</pre>	• Requirement: All 6 keys must be present. • Type Checking: Numeric floats/ints. • Bound Verification: - soil_ph: 0.0 to 14.0 - plot_area_ha > 0 - arrival_hour: 0 to 23 • Error Trigger: Any missing key generates explicit JSON error.

5. Wireframe Layout 2: Output JSON Response Interface

Annotated wireframe illustrating success and error outputs from the prediction module.

[CLI Output Wireframe - Response Payload]	Field Explanations
{ "regression_prediction": 4.12, "classification_prediction": 1, "classification_probability": 0.89, "cluster_label": 2, "group_code": "AI_A1_G08", "model_version": "1.0.0" }	1. regression_prediction: Yield target in Tons. 2. classification_prediction: Binary High/Low yield flag. 3. classification_probability: Model confidence score (0-1). 4. cluster_label: Assigned agricultural zone index. 5. Audit Tags: Group identifier and model iteration trace.

6. Responsible AI States & Edge-Case Handling

To comply with Responsible AI principles, the user experience must gracefully communicate errors, model limitations, and confidence thresholds.

System State	Trigger Condition	UI / System Response	Responsible AI Guardrail
Missing Key Error	Payload lacks required feature (e.g., <code>Content</code>)	HTTP 400 JSON Error Response	Prevents model hallucination on zero-input
Out-of-Bounds Input	Extremely unrealistic input (e.g., <code>input_val=25</code>)	Input Validation Error	Restricts predictions strictly within physical limits
Low Confidence Prediction	Classification probability between 0.45 and 0.55	Returns prediction with Low Confidence Warning	Warns user not to make critical financial decisions
Malformed JSON Payload	Syntax error in input string	Parses JSON and returns clear JSON error message	Avoids unexpected python crashes or server errors

7. Visual System Rationale & Design System

The visual and structural hierarchy of system outputs adheres to strict design guidelines for readability and integration into production logs.

Design Element	Choice / Standard	Rationale
Data Format	Standard JSON	Ensures language-agnostic integration for frontend dashboards and log parsers.
Color Codes (Terminal)	ANSI Color Traces (Green/Red)	Improves developer immediate recognition during automated test runs.
Numerical Precision	Floats rounded to 2 decimal places	Avoids false precision perception while maintaining scientific accuracy.
Error Transparency	Explicit list of missing keys	Eliminates developer guesswork during payload debugging.